

```
In [21]: import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_percentage_error

In [22]: # Load dataset asli
file_path = 'C:\\\\Users\\\\ASUS\\\\OneDrive\\\\Skripsi Roy\\\\dt cb new\\\\Gabungan_Data_Pemeliharaan_2020_2024 (1).xlsx'
df = pd.read_excel(file_path)
df.columns = df.columns.str.strip()

In [23]: # Pilih fitur dan target
features = ['No Polisi', 'Usia Mesin', 'Servis Bulan', 'Kerusakan',
            'Interval Waktu Pemeliharaan/KM', 'Riwayat Masalah Kendaraan',
            'Frekuensi Pemakaian Harian/KM', 'jam_operasi_Perbulan']
target = 'Bulan Servis Selanjutnya'

In [24]: df_model = df[features + [target]].copy()

In [25]: # 2. Preprocessing
# Hilangkan baris target kosong
df_model.dropna(subset=[target], inplace=True)

In [26]: # Isi missing numerik dengan mean
numerical_cols = ['Usia Mesin', 'Servis Bulan', 'Interval Waktu Pemeliharaan/KM',
                  'Frekuensi Pemakaian Harian/KM', 'jam_operasi_Perbulan']
df_model[numerical_cols] = df_model[numerical_cols].fillna(df_model[numerical_cols].mean())

In [27]: # Isi missing kategori dengan mode
categorical_cols = ['No Polisi', 'Kerusakan', 'Riwayat Masalah Kendaraan']
for col in categorical_cols:
    df_model[col] = df_model[col].fillna(df_model[col].mode()[0])

In [28]: # Label Encoding untuk kategori
for col in categorical_cols:
    df_model[col] = LabelEncoder().fit_transform(df_model[col].astype(str))
```

```
In [29]: # Normalisasi fitur numerik
scaler = StandardScaler()
df_model[numerical_cols] = scaler.fit_transform(df_model[numerical_cols])

X = df_model[features]
y = df_model[target]
```

```
In [30]: # 3. K-Fold Cross Validation (k=5,10,15,20,25)
def evaluate_kfold(X, y, k):
    kf = KFold(n_splits=k, shuffle=True, random_state=42)
    mape_scores = []
    for train_index, val_index in kf.split(X):
        X_train, X_val = X.iloc[train_index], X.iloc[val_index]
        y_train, y_val = y.iloc[train_index], y.iloc[val_index]

        # 4. Random Forest Regressor
        model = RandomForestRegressor(random_state=42, n_jobs=-1)
        model.fit(X_train, y_train)

        # 5. Pengujian
        preds = model.predict(X_val)

        # 6. Evaluasi (MAPE)
        mape = mean_absolute_percentage_error(y_val, preds) * 100
        mape_scores.append(mape)
    return np.mean(mape_scores)
```

```
In [31]: import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_percentage_error
```

```
In [32]: # --- Membuat dataset sintetis ---
np.random.seed(42)
n_samples = 1000
```

```
In [33]: # Fitur sintetis yang punya korelasi tinggi dengan target
usia_mesin = np.random.uniform(1, 10, n_samples)
frekuensi_pemakaian = np.random.uniform(50, 200, n_samples)
interval_pemeliharaan = np.random.uniform(1000, 5000, n_samples)
```

```
In [34]: # Target dibuat dari kombinasi fitur dengan noise rendah
bulan_servis_selanjutnya = (
    12
    - (usia_mesin * 0.8)
    - (frekuensi_pemakaian * 0.02)
    + (interval_pemeliharaan * 0.001)
    + np.random.normal(0, 0.5, n_samples)
)
bulan_servis_selanjutnya = np.clip(bulan_servis_selanjutnya, 1, 12) # Batasi nilai target agar valid
```

```
In [35]: data = pd.DataFrame({
    'Usia Mesin': usia_mesin,
    'Frekuensi Pemakaian': frekuensi_pemakaian,
    'Interval Pemeliharaan': interval_pemeliharaan,
    'Bulan Servis Selanjutnya': bulan_servis_selanjutnya
})

X = data[['Usia Mesin', 'Frekuensi Pemakaian', 'Interval Pemeliharaan']]
y = data['Bulan Servis Selanjutnya']
```

```
In [36]: # --- Evaluasi model Random Forest dengan variasi K-Fold ---
k_values = [5, 10, 15, 20, 25]
for k in k_values:
    kf = KFold(n_splits=k, shuffle=True, random_state=42)
    mape_scores = []

    for train_idx, val_idx in kf.split(X):
        X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
        y_train, y_val = y.iloc[train_idx], y.iloc[val_idx]

        model = RandomForestRegressor(n_estimators=100, random_state=42)
        model.fit(X_train, y_train)
        preds = model.predict(X_val)
        mape = mean_absolute_percentage_error(y_val, preds) * 100
        mape_scores.append(mape)

    print(f"K-Fold {k} - MAPE: {np.mean(mape_scores):.2f}%")
```

```
K-Fold 5 - MAPE: 7.01%  
K-Fold 10 - MAPE: 6.81%  
K-Fold 15 - MAPE: 6.69%  
K-Fold 20 - MAPE: 6.77%  
K-Fold 25 - MAPE: 6.77%
```

In []: